

Sequential and Binary Search

Two Strategies, and Why One Wins on Sorted Data

Unit III - Lecture 27

Sequential search check every element, $O(n)$

Binary search eliminate half each step, $O(\log n)$

The problem: finding one value among many

Given an array of n values and a key to find, return the key's INDEX if present, or -1 if not. The question today is not WHETHER this can be done, but HOW FAST.

- If the array could be in ANY order, there is only one honest option: check every element.
- But if the array is SORTED, a completely different strategy becomes possible - one that never has to look at most of the array at all.
- Both strategies are covered today, on the SAME 10-element array, so every number is directly comparable.

Sequential search: check every element, in order

```
int sequentialSearch(int arr[], int n,
                    int key)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
```

- No assumption about order - works on ANY array, sorted or not.
- Stops the moment it finds a match - the EARLY RETURN.
- If the key is not present, the loop runs all n times before giving up.

Best case: key is arr[0] - found in 1 comparison. Worst case: key is last, or missing - n comparisons.

Sequential search: verified on the same array

3	7	12	18	24	31	45	52	68	77
0	1	2	3	4	5	6	7	8	9

Searching for 45: checks indices 0,1,2,3,4,5,6 - found at index 6 after 7 comparisons.

key	sequential search cost	result
45	7 comparisons (indices 0-6)	found, index 6
77	10 comparisons (indices 0-9)	found, index 9 (worst case)
3	1 comparison (index 0)	found, index 0 (best case)
50	10 comparisons (entire array)	not found

Binary search: the one idea that changes everything

If the array is SORTED, checking the MIDDLE element tells you which HALF the key must be in - the other half can be thrown away without looking at a single value in it.

1. Look at the middle element.
2. Equal to the key? Done - found.
3. Middle SMALLER than the key? The key (if present) must be in the RIGHT half - discard the left half entirely.
4. Middle LARGER than the key? Discard the right half entirely, search only the left.
5. Repeat on whichever half remains, until found or the range is empty.

Binary search in C: low, high, mid

```
int binarySearch(int arr[], int n,
                 int key)
{
    int low = 0, high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

- low and high mark the CURRENT search range - it shrinks every iteration.
- $\text{mid} = (\text{low} + \text{high}) / 2$ always lands inside that range.
- $\text{low} = \text{mid} + 1$ or $\text{high} = \text{mid} - 1$ - never mid itself, since $\text{arr}[\text{mid}]$ was already ruled out.
- Loop ends when $\text{low} > \text{high}$ - the range became empty, key is not present.

Binary search: verified trace, searching for 45

3	7	12	18	24	31	45	52	68	77
---	---	----	----	----	----	----	----	----	----

Step	low	high	mid	arr[mid]	Action
1	0	9	4	24	24 < 45, search right
2	5	9	7	52	52 > 45, search left
3	5	6	5	31	31 < 45, search right
4	6	6	6	45	MATCH - found at index 6

4 comparisons for 10 elements - sequential search needed 7 for the same key.

Searching for 50 (not present) takes 5 steps before low > high - still far fewer than checking all 10 elements.

Complexity, side by side

Aspect	Sequential search	Binary search
Precondition	None - any order	MUST be sorted first
Best case	$O(1)$	$O(1)$
Average case	$O(n)$	$O(\log n)$
Worst case	$O(n)$	$O(\log n)$
1,000 elements, worst case	up to 1,000 comparisons	up to about 10 comparisons
1,000,000 elements, worst case	up to 1,000,000 comparisons	up to about 20 comparisons

The catch: binary search is not free

If the data is already sorted

Binary search wins outright. No reason to ever use sequential search on sorted data for repeated lookups.

If the data is NOT sorted yet

Sorting first costs $O(n \log n)$ (Unit IV). Worth it only if you will search MANY times - a single one-off lookup is cheaper with plain sequential search.

- Binary search also needs an ARRAY with fast random access to the middle - it does not translate directly to a plain linked list, where reaching 'the middle' is itself $O(n)$.
- This limitation is exactly why binary search TREES (Lecture 28) matter - a tree gives the halving benefit of binary search while staying easy to insert into, without needing a sorted array at all.

Sorted data changes the rules of searching

- Sequential search: check every element in order, $O(n)$ worst case, works on any array.
- Binary search: check the middle, discard half, repeat - $O(\log n)$ worst case, but requires a SORTED array.
- Verified on the same 10-element array: 7 comparisons (sequential) vs 4 comparisons (binary) to find the same key.
- The gap widens fast: 1,000,000 elements needs up to 1,000,000 sequential comparisons but only about 20 binary comparisons.
- Binary search needs random access to the middle - the next lecture's binary search TREE gets the same speed without that requirement.

EXIT TICKET Array: 5 11 19 23 30 37 41 49. Trace binary search for 23, showing low/high/mid at every step.